Experiment No: 4

Name: Dipesh Makdiya

Class: D15C

Roll No: 32

Experiment:

K-Nearest Neighbours (KNN) for Classification

---

1. Dataset Source

Dataset Used: Iris Dataset

Type: Classification

Source:
https://scikit-learn.org/stable/datasets/toy_dataset.html#iris-dataset

Access Method:
```
from sklearn.datasets import load_iris
```

Justification:

- Standard real-world benchmark dataset
- Well-suited for distance-based classifiers
- Small and clean dataset for easy performance evaluation

---

2. Dataset Description

2.1 Iris Dataset

Objective:
To classify iris flowers into different species using the K-Nearest Neighbours algorithm.

Number of Instances: 150
Number of Features: 4

Features:

- Sepal length
- Sepal width
- Petal length
- Petal width

Target Variable:

Species:

- Setosa
- Versicolor
- Virginica

Characteristics:

- Multiclass classification problem
- Numerical feature values
- No missing data
- Suitable for distance-based algorithms

---

3. Mathematical Formulation of the Algorithm

3.1 K-Nearest Neighbours (KNN)

KNN is a **non-parametric and instance-based learning algorithm** that classifies a data point based on the majority class among its nearest neighbours.

Euclidean Distance:

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Where:

- xx = test sample
- yy = training sample
- nn = number of features

```
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
print(confusion_matrix(y_test, y_pred_rf))
print(classification_report(y_test, y_pred_rf))
```

```
Random Forest Accuracy: 1.0
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

Classification Rule:

1. Select the value of **K**
2. Compute the distance between the test data and all training samples
3. Identify the **K nearest neighbours**
4. Assign the class by **majority voting**

$$\hat{y} = mode(y_1, y_2, ..., y_k)$$

---

4. Algorithm Limitations

KNN Limitations

- Computationally expensive for large datasets
- Requires feature scaling for better performance
- Sensitive to irrelevant and noisy features
- Performance depends onthe optimal selection of **K**
- High memory requirement (stores entire dataset)

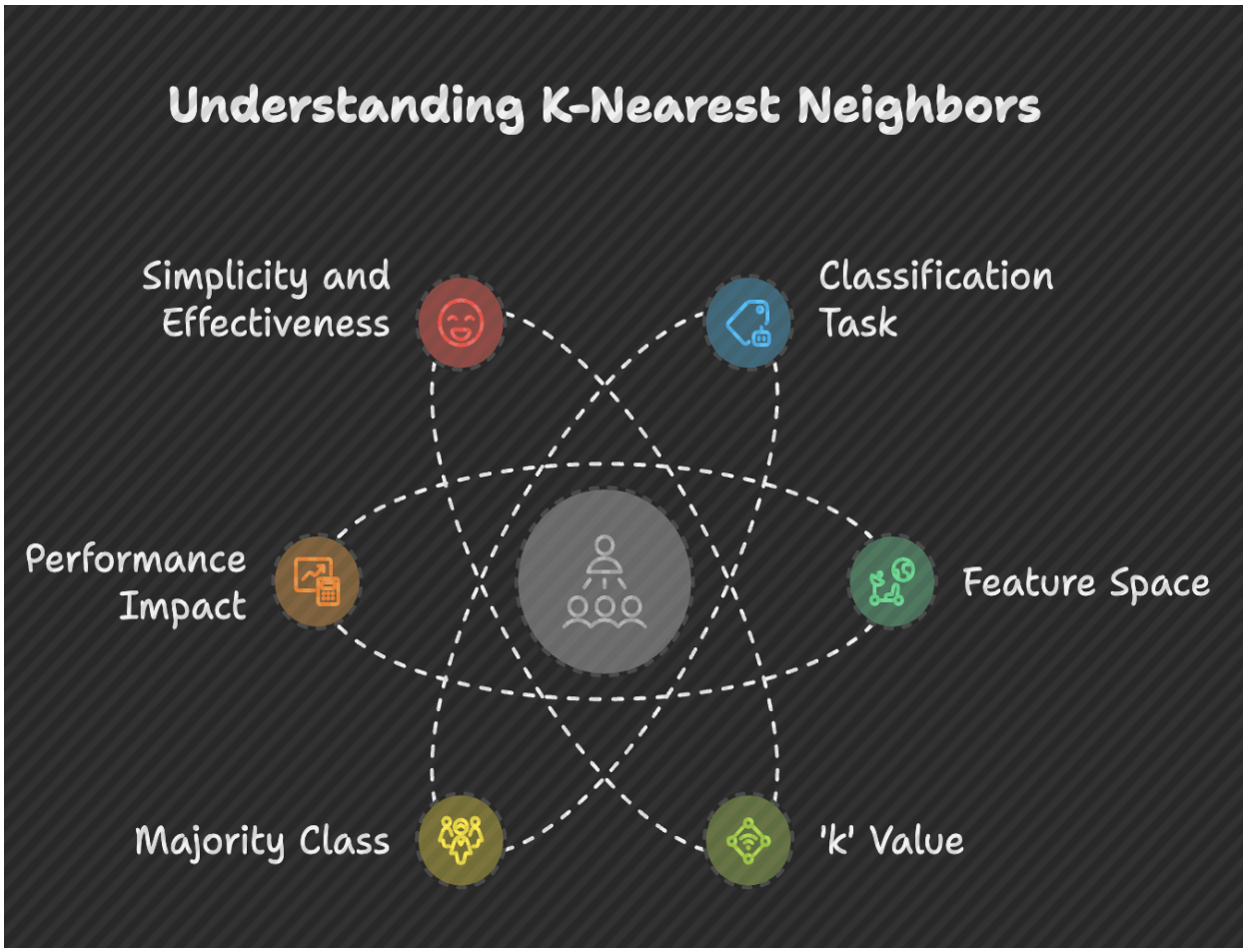
---

5. Methodology / Workflow

Step-by-Step Workflow:

1. Import required libraries
2. Load the Iris dataset
3. Data exploration
4. Feature scaling using StandardScaler
5. Train-test split
6. Train the KNN classifier
7. Perform prediction
8. Evaluate model performance
9. Hyperparameter tuning for optimal K value

Workflow Diagram:



Dataset → Feature Scaling → Train/Test Split →
Model Training → Prediction → Evaluation

---

6. Performance Analysis before Tuning

Evaluation Metrics Used:

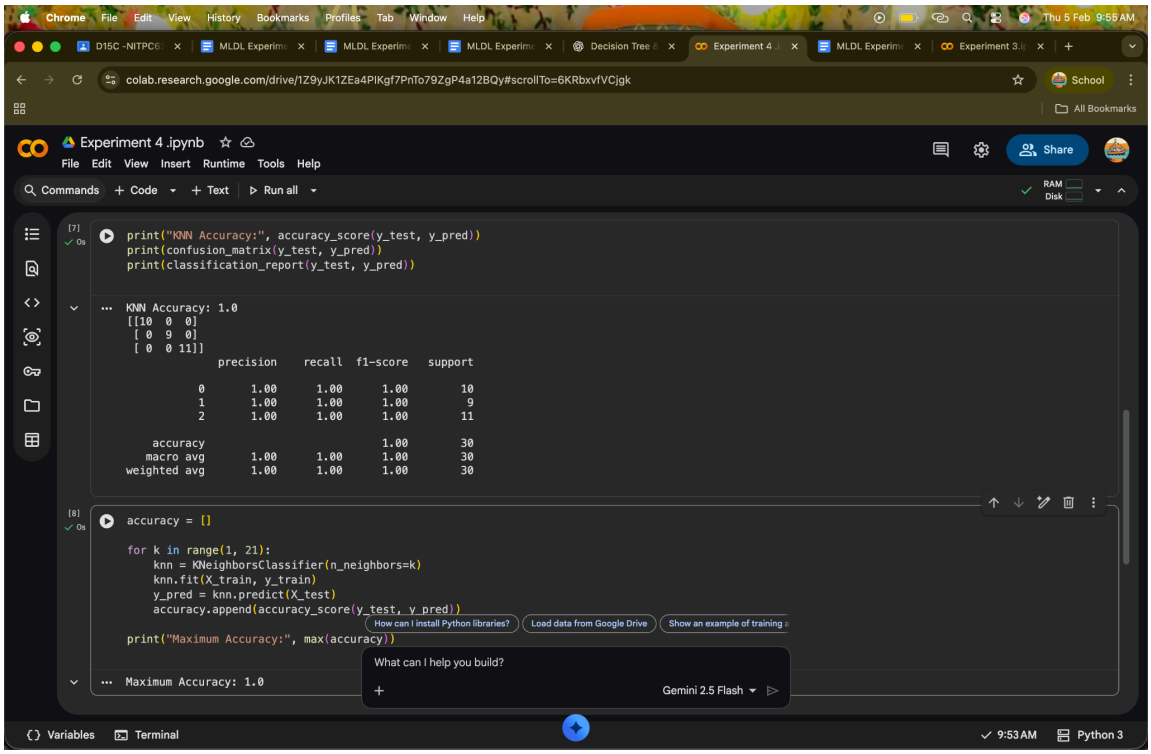
- Accuracy
- Confusion Matrix
- Classification Report
    - Precision
    - Recall

- F1-score

Interpretation:

- Higher accuracy indicates better classification
- The confusion matrix shows correct and incorrect predictions
- Precision indicates the correctness of the predicted class
- Recall indicates the model's ability to identify the actual class
- F1-score provides a balance between precision and recall

Optimal selection of **K** improves:

- Model accuracy
- Generalization ability
- Stability of predictions
- 

---

7. Results After Hyperparameter Tuning

Tuned Parameter:

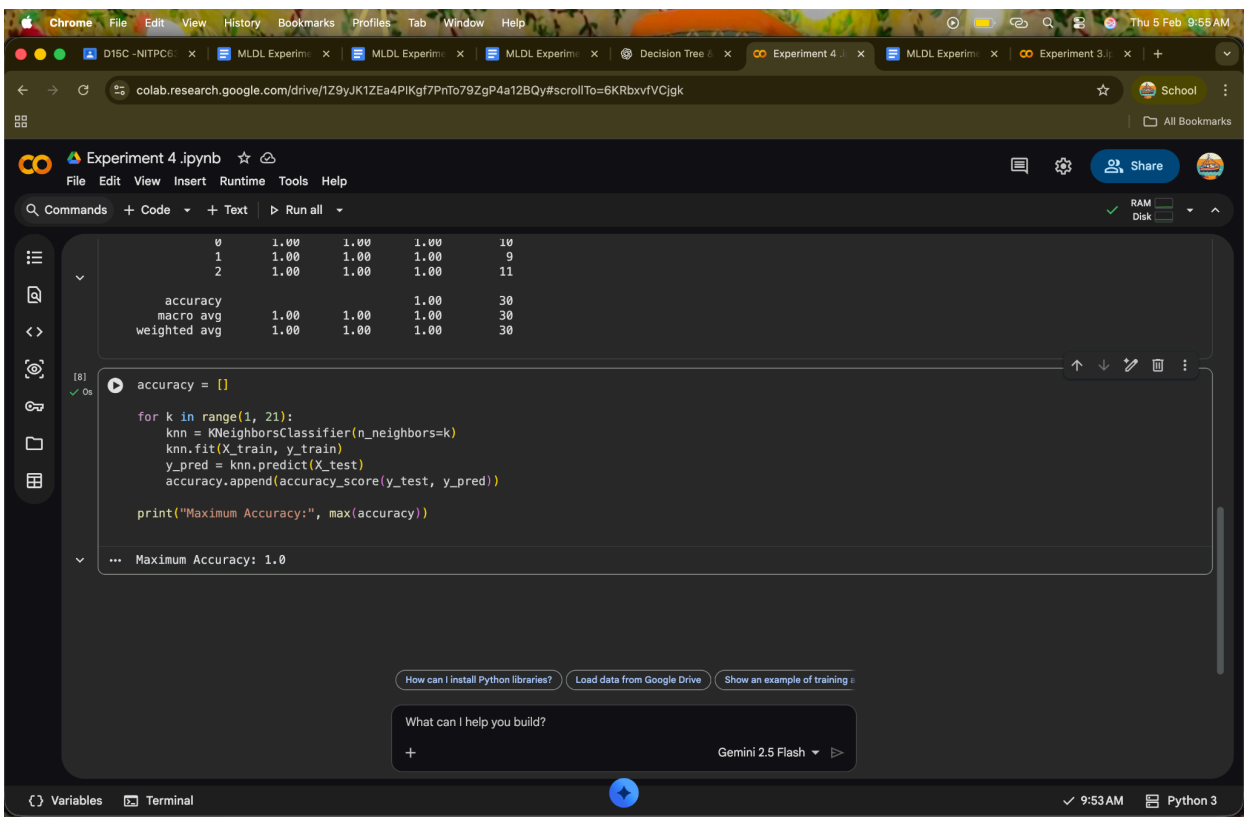
- Number of neighbours (K)

Impact of K:

K Value	Effect
Small K	Overfitting, sensitive to noise
Large K	Underfitting, smoother decision boundary
Optimal K	Best bias-variance balance

Other Tuned Parameters:

- Distance metric (Euclidean, Manhattan)
- Weights (uniform/distance)



---

8. Google Colab Link

Experiment 4 .ipynb

---

9. Result

K-Nearest Neighbours achieved high classification accuracy on the Iris dataset.
Model performance varied with different values of K, and moderate K values provided the best results.

---

10. Conclusion

The KNN algorithm was successfully implemented for multiclass classification using the Iris dataset.
Feature scaling and optimal selection of K significantly improved the model performance.
This experiment demonstrates that KNN is an effective algorithm for small to medium-sized datasets and distance-based classification problems.